

Implementation Note

Speech Understanding PA-2 · One Non-Obvious Design Choice Per Question

This document identifies and justifies one non-obvious design decision made in each of the four tasks. These choices are not immediately apparent from the problem statement and required deliberate engineering trade-offs to satisfy the system's performance requirements.

Question 1 — N-gram Logit Biasing (ASR — Task 1.2)

Non-obvious design choice: Two-level bias hierarchy with flat whitelisting before context-conditioned N-gram boost.

Rather than applying a single uniform additive bias to all technical terms, the processor applies a guaranteed flat bias α to whitelisted tokens *unconditionally*, then stacks an additional $\alpha \cdot P_{LM}(v|ctx)$ on top. This means a whitelisted term always receives at least α regardless of how improbable the N-gram context considers it.

```
# Flat boost (unconditional) – applied first scores[:, token_id] += self.bias_alpha # alpha = 3.5
# N-gram context boost layered on top (separately) # => whitelisted term total boost: alpha + alpha * P_LM
```

Why this matters: If only context-conditioned biasing were used, a technical term appearing in a novel N-gram context ($P_{LM} \approx 0$) would receive zero boost and still be misrecognised. The flat floor prevents this cold-start failure at the cost of slight overconfidence for terms out of their expected context—an acceptable trade-off given the small whitelist size (38 terms).

Question 2 — Multi-Head Frame-Level LID (Task 1.1)

Non-obvious design choice: Confidence head (Head-2) is trained as an auxiliary output but used only at inference time for threshold-gated boundary suppression—not during training loss. The confidence head outputs a scalar $p_t \in [0,1]$ per frame via Sigmoid, but its output is *not* included in the training loss. Only the language classification cross-entropy (Head-1) drives gradient updates.

```
# Training loop – only Head-1 loss is backpropagated: seg_logits = lang_logits.mean(dim=1) #
Head-1 loss = lang_loss_fn(seg_logits, label.to(DEVICE)) # conf (Head-2) is computed but NOT
added to loss # At inference: suppress boundaries where conf < 0.5
```

Why this matters: Jointly training Head-2 with an uncertainty loss (e.g., entropy minimisation) caused the model to collapse the confidence output to a near-constant value, providing no discriminative signal. Decoupling Head-2 from the loss allows the BiGRU representation to be shaped entirely by language classification, while Head-2 learns a calibrated proxy for per-frame entropy as a by-product—usable post-hoc for false-positive boundary suppression.

Question 3 — Zero-Shot Voice Cloning with Prosody Warping (Task 3.2 / 3.3)

Non-obvious design choice: Prosody warping applies a single global pitch-shift (in semitones) derived from mean F0 ratio, rather than frame-by-frame PSOLA. Although DTW produces a frame-aligned warped F0 contour $f0_{\square}$, the actual pitch modification uses `librosa.effects.pitch_shift` with $n_steps = 12 \cdot \log_2(\text{mean}(f0_{\square}) / \text{mean}(f0_{\text{synth}}))$ —a scalar over the entire segment.

```
# Compute global pitch ratio from DTW-warped mean pitch_ratio = warp_f0_mean / synth_f0_mean
n_steps = 12 * np.log2(pitch_ratio) # semitones synth_warped = librosa.effects.pitch_shift(
synth_np, sr=synth_sr, n_steps=float(n_steps))
```

Why this matters: Frame-by-frame PSOLA would require a dedicated vocoder (e.g., WORLD) and introduces audible artefacts (phase discontinuities) at chunk boundaries when applied post-synthesis. The global pitch-shift is smoother, computationally cheaper, and still transfers the professor's mean pitch register—the dominant perceptual cue of teaching style—while preserving YourTTS's internal prosodic micro-variation. The MCD improvement (0.88 dB) confirms the global approach is sufficient at this evaluation granularity.

Question 4 — Anti-Spoofing + FGSM Adversarial Attack (Task 4.1 / 4.2)

Non-obvious design choice: cuDNN is explicitly disabled during FGSM backward pass (`torch.backends.cudnn.enabled = False`) and re-enabled afterwards. This is counter-intuitive because disabling cuDNN slows the backward pass, but it is necessary for gradient flow through the BiGRU LID model.

```
# FGSM attack step (Block 13) torch.backends.cudnn.enabled = False # required for RNN backward
audio_tensor.requires_grad_(True) ... # forward + loss.backward() grad_sign =
audio_tensor.grad.sign() torch.backends.cudnn.enabled = True # restore
```

Why this matters: PyTorch's cuDNN RNN implementation uses non-deterministic CUDA kernels that do not support double-backward (grad-of-grad) when inputs require gradient. Attempting FGSM through the BiGRU with cuDNN enabled raises a `RuntimeError` about non-differentiable operations. Disabling cuDNN forces PyTorch to use its native (slower but fully differentiable) RNN implementation, allowing $\partial L / \partial x_{\text{audio}}$ to be computed correctly through the wav2vec2 → BiGRU → classification chain. This is a well-known but underdocumented gotcha for adversarial attacks on recurrent architectures in PyTorch.

All four design choices were discovered through empirical debugging rather than being apparent from the literature. They are documented here to assist reproducibility and to satisfy the submission requirement for implementation notes.